

Varios estudiantes se ha titulado y están dispuesto a encontrar trabajo. Para esto, deciden postular a lo más a N trabajos (hay solamente N disponibles). Sin embargo, los postulantes sólo pueden quedar en un trabajo, el primero en el que sea aceptado cuando cierre la postulación. Por otro lado, para que un estudiante pueda postular a un trabajo, su postulación haber sido abierta.

A su lado derecho, primero se muestran las variables globales, junto con a la función *iniciarPostulaciones* que se llama cada vez que se inicializan las N postulaciones (se puede volver a iniciar una proceso de postulación). *DestruirPostulaciones* se llama cuando todos los trabajos ya han retornado y se encarga de liberar memoria en caso de ser necesario. En el caso de las variables globales, *priQueue* es puntero a un arreglo de listas de prioridades que ayuda a ver los postulantes a los N trabajos ordenados por su *ranking*. Por otro lado, *postulacionTrabajos* corresponde a un arreglo que tendrá los id de los estudiantes que hayan conseguido el trabajo.

Más abajo, se da una implementación de la función *postularTrabajo(Estudiante *est, int *preferencias, double *rank)*, la cual es llamada por cada estudiante (un thread por estudiante), donde los parámetros corresponden a un puntero a un estudiante (ver estructura más abajo), un puntero a un arreglo de tamaño N con valores 0 en los índices de los trabajos donde no quiere postular y 1 donde sí quiere, y por último un puntero a un arreglo *double* donde estarán el ranking con el cual postulará. En la variable *trabajo_id* queda el número de trabajo encontrado (asuma que siempre encontrará uno).

Por último se muestra la implementación de *cerrarPostulacion(int i)*, la cuál es llamada por cada thread para cerrar el trabajo i-ésimo, donde se modifica la variable global *trabajo_finalizado* en el índice *i* y se retorna el *id* del estudiante que obtuvo el trabajo. *cerrarPostulacion* **no retorna hasta** que un alumno haya obtenido el trabajo.

```
typedef struct {
    int id;
    int trabajo_id;
} Estudiante;

typedef struct {
    Estudiante *est;
    double *ranking;
} Postulacion;
```

```
PriQueue *priQueue[N];
int postulacionTrabajos[N];

void iniciarPostulaciones(){
    for (int i = 0; i < N; i++) {
        priQueue[i] = makePriQueue();
        postulacionTrabajos[i] = -1;
    }
}

void destruirPostulaciones(){
    for (int i = 0; i < N; i++)
        destroyPriQueue(priQueue[i]);
}

void postularTrabajo(
    Estudiante *est, int *preferencias, double *rank) {
    est->trabajo_id = -1;
    Postulacion postulacion = {est, rank};
    for (int i = 0; i < N; i++) {
        if (postulacionTrabajos[i]==-1 && preferencias[i])
            priPut(priQueue[i], &postulacion, rank[i]);
    }
    while (est->trabajo_id== -1)
        ;
}

int cerrarPostulacion(int i) {
    while (postulacionTrabajos[i]==-1) {
        while(!emptyPriQueue(priQueue[i])){
            Postulacion *ppostulacion = priGet(priQueue[i]);
            if (ppostulacion->est->trabajo_id == -1) {
                postulacionTrabajos[i] = ppostulacion->est->id;
                ppostulacion->est->trabajo_id = i;
                for(int j = 0; j < N; j++)
                    priDel(priQueue[j], ppostulacion);
                break; // SE AGREGÓ ESTE BREAK !
            }
        }
    }
    return postulacionTrabajos[i];
}
```

Implemente nuevamente las funciones ***iniciarPostulaciones***, ***destruirPostulaciones***, ***postularTrabajo*** y ***cerrarPostulacion*** (manteniendo la funcionalidad de estas), de tal manera que se eviten *datarace* y *deadlocks*, no se preocupe de la hambruna. Usted puede crear más variables globales. Además usted puede agregar más parámetros en la estructura *Postulacion*.

Para programar la sincronización requerida Ud. debe usar spin-locks. No puede usar otras herramientas de sincronización como mutex/condiciones, semáforos o mensajes.

Note que N es una variable definida en *postulacion.h*. Se le recomienda probar con N pequeños pero mayores a 6 para *debuggear* su tarea. Sin embargo, la entrega debe ser con el N definido en el archivo original.

Prueba de la tarea: La verificación del funcionamiento correcto de su tarea se realizará primero usando pthreads y verdaderos spin-locks que esperan con busy-waiting. Por lo tanto, se ocupa el 100% de la CPU. Después su tarea se verificará usando pthreads y spin-locks funcionalmente equivalentes pero implementados en base a mutex/condiciones.

Instrucciones

Descargue *t6.zip* de U-cursos y descomprímalo. Ejecute el comando *make* sin parámetros en el directorio *T6* para recibir instrucciones acerca del archivo en donde debe programar su solución (*postulaciones.c*), cómo compilar, probar y depurar su solución, los requisitos que debe cumplir para aprobar la tarea y cómo entregar su tarea por U-cursos.

Entrega

Ud. solo debe entregar por medio de U-cursos el archivo *postulaciones.zip* generado por *make zip*. Este incluye *postulaciones.c* y *resultados.txt*. Recuerde descargar el archivo que subió, descargar nuevamente los archivos adjuntos y volver a probar la tarea tal cual como la subió a U-cursos. Solo así estará seguro de no haber entregado archivos incorrectos. Se descuenta medio punto por día de atraso. No se consideran los días de receso, sábado, domingo o festivos.